

0x03 Ядро Linux. Обзор ядра Linux и его архитектуры. Основные функции ядра.

Разработка ядра Linux

**Меня хорошо видно
& слышно?**



Правила вебинара



Активно
участвуем



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

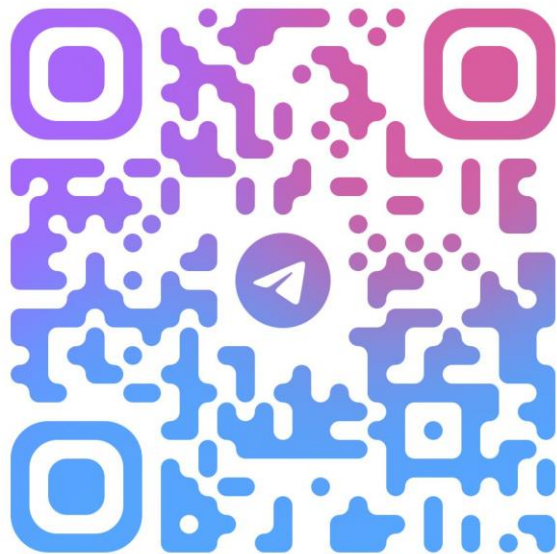
Правила вебинара



Off-topic обсуждаем
в закрытом чате в
Telegram !

***(также обсудить Off-topic будет**
возможность в конце занятия
после отключения записи)

Кто еще не
присоединился,
сканируйте QR код :)



Цели вебинара

По завершении занятия вы будете знакомы с

Ядром Linux и его архитектурой

Процессом загрузки ядра

Основными функциями ядра Linux



**Кто-нибудь может что-то
рассказать об архитектуре
ядра Linux?**



Маршрут вебинара

Дерево каталогов Linux

Архитектура ядра Linux

Процесс загрузки ядра Linux

Основные функции ядра Linux



Дерево каталогов Linux

Дерево каталогов Linux

Иерархическая структура, в которой каждый каталог содержит специфические для ядра файлы и подкаталоги

- Обеспечивает логическую организацию кода и ресурсов
- Позволяет разработчикам легко находить и модифицировать необходимые компоненты

Дерево каталогов Linux



Мы начнем с изучения **основных каталогов**, таких как arch, block, drivers, fs, kernel, mm, net и других, а также их назначения и содержания. Каждый из этих каталогов играет **важную роль в функционировании ядра** и его взаимодействии с аппаратным обеспечением и пользовательскими приложениями.



Кроме того, мы обсудим **архитектурные компоненты ядра**, которые организованы в этом дереве каталогов, и как они способствуют общей работе системы. Это знание будет полезно для дальнейшего изучения и разработки в области ядра Linux.



Давайте начнем наше путешествие по дереву каталогов ядра Linux и откроем для себя его структуру и функциональность!

Демонстрация таблицы в лекции

Архитектура ядра Linux

Монолитная архитектура ядра Linux

- один из основных подходов к проектированию операционных систем
- **все основные функции ядра**, включая управление процессами, памятью, устройствами и системными вызовами, реализованы в **одном большом исполняемом файле**

Давайте вспомним основные аспекты монолитной архитектуры ядра Linux



Основные характеристики архитектуры ядра Linux:

Единое ядро

- Все компоненты ядра, такие как планировщик процессов, управление памятью, файловая система и драйверы устройств, находятся в одном исполняемом файле
- Ядро работает более эффективно
- Все функции могут взаимодействовать друг с другом без необходимости в сложных механизмах межпроцессного взаимодействия



Основные характеристики архитектуры ядра Linux:

Высокая производительность

- Все компоненты ядра находятся в одном адресном пространстве
- Вызовы функций между ними происходят быстрее, чем в микроядерных архитектурах, где требуется межпроцессное взаимодействие
- Монолитное ядро более производительнее для задач, требующих высокой скорости обработки



Основные характеристики архитектуры ядра Linux:

Сложность разработки

- Все компоненты ядра находятся в одном исполняемом файле, что усложняет разработку и отладку ядра Linux.
- Высокая взаимозависимость между модулями
- Затрудняется выявление и исправление ошибок
- Увеличивается риск влияния одной ошибки на стабильность всей системы



Основные характеристики архитектуры ядра Linux:

Большой размер

- Ядро может стать очень большим и сложным
- Может затрудниться его поддержка и развитие, особенно в больших проектах
- Ошибки в одном компоненте могут повлиять на стабильность всего ядра

Основные характеристики архитектуры ядра Linux:

Безопасность

- Поскольку все компоненты ядра работают в одном адресном пространстве
- Уязвимости в одном из компоненте могут привести к компрометации всей системы
- Требуется тщательная проверка и тестирование кода ядра



Основные характеристики архитектуры ядра Linux:

Загрузка модулей

- Несмотря на то, что ядро Linux является монолитным, оно поддерживает загрузку модулей в режиме выполнения
- Это позволяет добавлять или удалять драйверы и другие компоненты без необходимости перезагрузки системы
- Модули могут быть загружены по мере необходимости, что делает систему более гибкой



Процесс загрузки ядра Linux

Процесс загрузки ядра

Загрузка ядра Linux – **сложный многоэтапный процесс**, включающий взаимодействие различных компонентов системы

Давайте рассмотрим его подробно



Процесс загрузки ядра

Предварительная загрузка

Процесс начинается с BIOS/UEFI:

- BIOS выполняет самотест (POST)
- Определяет доступные устройства
- Находит загрузочные устройства
- Читает загрузочный сектор (MBR/GPT)



Процесс загрузки ядра

Загрузчик (Bootloader)

После обнаружения загрузочного сектора:

- GRUB загружается в память
- Отображается меню выбора ядра
- Пользователь выбирает нужную конфигурацию
- Загрузчик читает образ ядра



Процесс загрузки ядра

Распаковка ядра

После загрузки образа ядра происходит:

- Декомпрессия сжатого кода
- Перемещение в верхнюю часть адресного пространства
- Подготовка к выполнению



Инициализация ядра

Базовая настройка:

- Инициализация CPU
- Настройка памяти
- Подготовка стеков



Инициализация ядра

Регистры и прерывания:

- Настройка таблицы векторов прерываний (IDT)
- Инициализация обработчиков исключений
- Регистрация системных вызовов



Инициализация ядра

Управление памятью:

- Подготовка структуры страниц памяти
- Настройка механизмов виртуальной памяти
- Аллокация первичных структур данных

Инициализация ядра

Инициализация планировщика и таймеров:

- Инициализация планировщика задач
- Запуск системного таймера
- Настройка механизмов синхронизации

Инициализация ядра

Ввод/вывод:

- Регистрация драйверов устройств
- Инициализация консоли
- Подготовка файловых систем

Инициализация ядра

Запуск Процессов пользовательского уровня:

- Запуск init-процесса
- Инициализация системных служб
- Подготовка к работе с пользователем

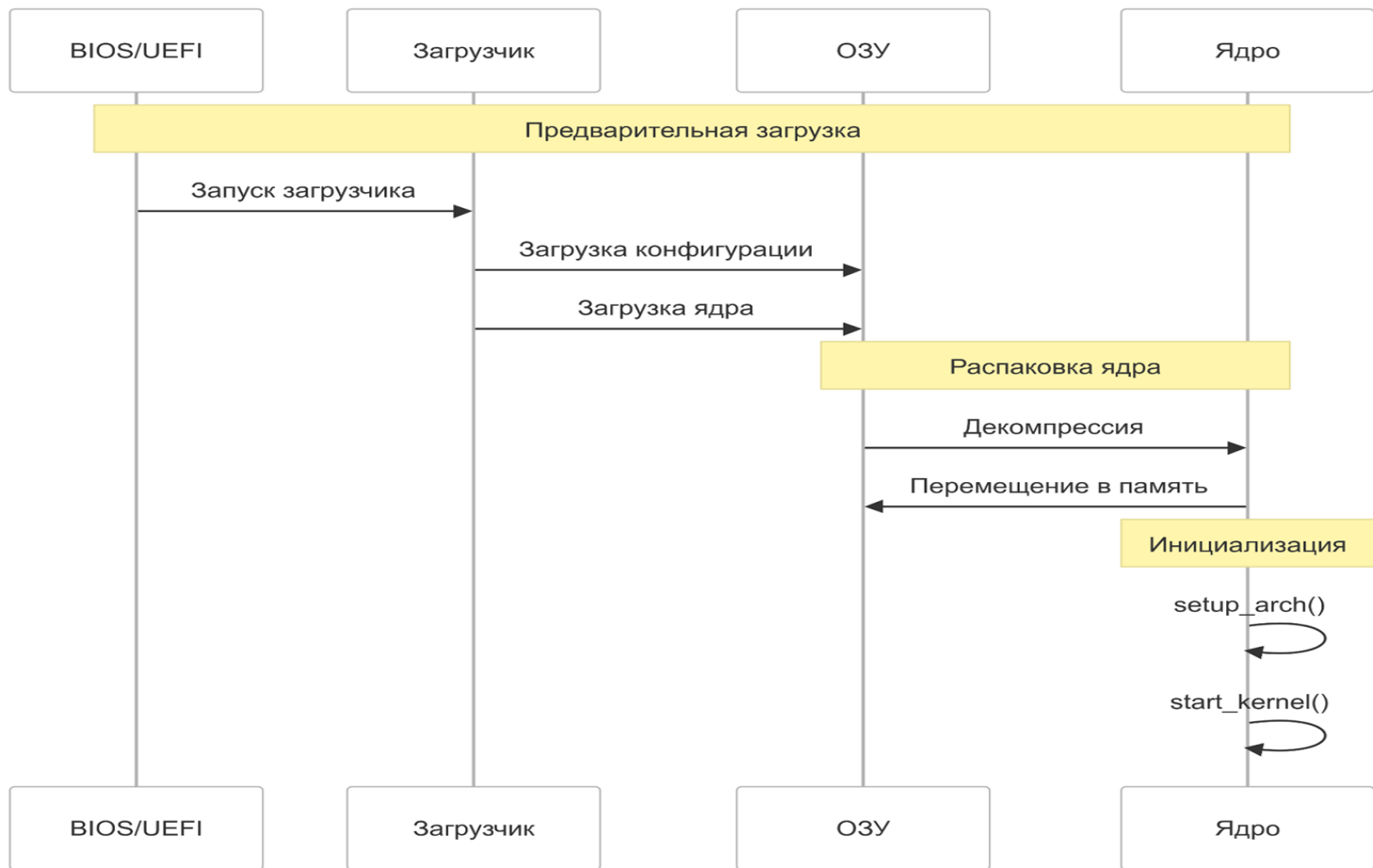


Инициализация ядра

Окончательные проверки:

- Верификация состояния системы
- Настройка сетевых компонентов
- Проверка доступности ресурсов





Итог

На диаграмме показаны ключевые этапы загрузки, где:

- `start_kernel()` запускает основной процесс инициализации, внутри вызывается `setup_arch()` отвечает за настройку архитектурно-зависимых компонентов и др. важные функции для инициализации ядра
- Все операции выполняются в реальном режиме до включения защищенного режима



Итог

После завершения всех этих этапов система готова к работе и может обрабатывать запросы пользователя

Каждый этап критически важен для корректной работы системы, и нарушение последовательности может привести к сбоям или некорректной работе



Попробуем закопаться

Изучаем функцию `start_kernel()`

Опустим часть с загрузчиком, загрузкой ядра и декомпрессией, потому что там слишком много низкоуровневого кода и его рассмотрение займет очень много времени.

Саму инициализацию будем рассматривать с функции `start_kernel()`.

Изучаем функцию start_kernel()

Начнем с объявления функции и локальных переменных на стеке

```
asmlinkage __visible void __init __no_sanitize_address start_kernel(void)
{
    char *command_line;
    char *after_dashes;
```



Изучаем функцию `start_kernel()`

Объявление функции

- `asmlinkage`: функция может принимать аргументы через стек, что важно для совместимости с вызовами из ассемблера
- `__visible`: функция видима для других модулей и может быть вызвана из других частей кода
- `__init`: функция используется только во время инициализации и может быть удалена из памяти после завершения инициализации
- `__no_sanitize_address`: указывает компилятору не применять проверки на наличие ошибок в адресах памяти для этой функции



Изучаем функцию `start_kernel()`

```
char *command_line;  
char *after_dashes;
```

- `command_line`: указатель на строку, содержащую командную строку, переданную ядру при загрузке
- `after_dashes`: указатель, который будет использоваться для хранения части командной строки после символов `--`, если они присутствуют

Пример из `/boot/grub.cfg`:

```
linux          /vmlinuz-6.1.0-29-amd64 root=/dev/mapper/host--vg-root ro single
```



Инициализация стека задачи

```
set_task_stack_end_magic(&init_task);
```

- `set_task_stack_end_magic(&init_task)`: устанавливает специальное значение (магическое число) в конце стека задачи `init_task` для обнаружения переполнения стека
- Это помогает предотвратить ошибки, связанные с переполнением стека



Инициализация стека задачи

После инициализации `init_task` ядро создает процесс `init` (обычно с `pid == 1`)

- Процесс `init` отвечает за запуск других пользовательских процессов и инициализацию системы
- Процесс `init` (или его современный аналог, такой как `systemd`) управляет жизненным циклом всех пользовательских процессов



Настройка идентификатора процессора

```
smp_setup_processor_id();
```

- `smp_setup_processor_id()`: настраивает идентификаторы процессоров для системы с многопроцессорной архитектурой (SMP)
- Это важно для правильной работы с несколькими процессорами



Настройка идентификатора процессора

В архитектуре `x86_64` функция `smp_setup_processor_id()` обычно имеет пустое тело, так как идентификаторы процессоров инициализируются в других частях кода

Однако, она может быть определена как заглушка для совместимости с другими архитектурами



Инициализация идентификатора сборки

```
init_vmlinux_build_id();
```

- `init_vmlinux_build_id()`: инициализирует идентификатор сборки ядра
 - может использоваться для отслеживания версии ядра и отладки
 - вшивается на этапе компиляции vmlinux



Инициализация групп контроля

```
cgroup_init_early();
```

- `cgroup_init_early()`: инициализирует группы контроля (cgroups) на раннем этапе загрузки
- Cgroups позволяют ограничивать и управлять ресурсами, используемыми процессами



Отключение прерываний

```
local_irq_disable();  
early_boot_irqs_disabled = true;
```

- `local_irq_disable()`: отключает прерывания на текущем процессоре. Это необходимо для выполнения критически важных операций без прерываний.
- `early_boot_irqs_disabled = true;` - устанавливает флаг, указывающий, что прерывания отключены на раннем этапе загрузки.



Отключение прерываний

```
static __always_inline void native_irq_disable(void)
{
    asm volatile("cli": : : "memory");
}
```

- Под капотом вызывается `cli`: отключает прерывания на процессоре, устанавливая флаг прерываний (IF) в 0.
- Процессор не будет реагировать на аппаратные прерывания, пока они не будут снова включены с помощью команды `sti` (Set Interrupt Flag).

Инициализация процессора

```
boot_cpu_init();
```

- `boot_cpu_init()`: выполняет инициализацию процессора, который загружает ядро
- Сохраняет `cru_id` для первого процессора в системе, устанавливает состояния процессора



Инициализация адреса страниц

```
page_address_init();
```

- `page_address_init()`: инициализирует массив `page_address_htable`, который представляет собой хэш-таблицу для управления адресами страниц.

```
void __init page_address_init(void)
{
    int i;

    for (i = 0; i < ARRAY_SIZE(page_address_htable); i++) {
        INIT_LIST_HEAD(&page_address_htable[i].lh);
        spin_lock_init(&page_address_htable[i].lock);
    }
}
```

Вывод баннера ядра

```
pr_notice("%s", linux_banner);
```

- `pr_notice("%s", linux_banner)`: выводит баннер ядра (информацию о версии и сборке) в dmesg
- Это полезно для идентификации версии ядра при загрузке



Вывод баннера ядра

- Пример баннера:

```
Linux version 6.1.0-32-amd64 (debian-kernel@lists.debian.org) (gcc-12 (Debian  
12.2.0-14) 12.2.0, GNU ld (GNU Binutils for Debian) 2.40) #1 SMP  
PREEMPT_DYNAMIC Debian 6.1.129-1 (2025-...)
```



Настройка архитектурных параметров

```
setup_arch(&command_line);
```

- `setup_arch(&command_line):`
 - настраивает память
 - инициализирует KASLR и KASAN, раннюю таблицу прерываний процессоры и другие архитектурные особенности
 - обрабатывает командную строку, переданную ядру, для настройки специфичных для архитектуры параметров

Настройка параметров конфигурации загрузки

```
setup_boot_config();
```

- `setup_boot_config()`: настраивает параметры, специфичные для системы, такие как настройки аппаратного обеспечения и конфигурации, которые могут быть заданы в командной строке.



Обработка командной строки

```
setup_command_line(command_line);
```

- `setup_command_line(command_line):`
 - разбирает аргументы командной строки, переданные ядру
 - устанавливает значения для различных переменных, которые будут использоваться в процессе загрузки

Установка количества доступных процессоров

```
setup_nr_cpu_ids();
```

- `setup_nr_cpu_ids()`:
 - определяет, сколько процессоров будет использоваться в многопроцессорной системе
 - инициализирует соответствующие структуры данных

Инициализация областей памяти для каждого процессора

```
setup_per_cpu_areas();
```

- `setup_per_cpu_areas()`: настраивает области памяти, выделенные для каждого процессора, чтобы хранить информацию, специфичную для каждого процессора в многопроцессорных системах



Подготовка загрузочного процессора для многопроцессорных систем

```
setup_per_cpu_areas();
```

- `smp_prepare_boot_cpu()`: выполняет архитектурные специфические действия, такие как инициализация кэшей или других аппаратных компонентов, необходимых для работы процессора



Инициализация механизмов "горячей" подмены процессоров

```
boot_cpu_hotplug_init();
```

- `boot_cpu_hotplug_init()`: позволяет добавлять или удалять процессоры из системы без перезагрузки, что полезно для управления ресурсами в реальном времени



Создание списков зон памяти

```
build_all_zonelists(NULL);
```

- `build_all_zonelists(NULL)`:
 - создает списки зон памяти, которые используются для управления памятью в системе
 - представляет разные области памяти, такие как память для страниц, выделенных для ядра, и память для пользовательских процессов

Инициализация системы управления страницами памяти

```
page_alloc_init();
```

- `page_alloc_init()`: подготавливает ядро к корректной работе с выделением страниц памяти с учётом динамического добавления и удаления процессоров
- Это важно для производительности и корректности работы подсистемы памяти в многопроцессорных системах



Вывод командной строки ядра в системный журнал

```
pr_notice("Kernel command line: %s\n", saved_command_line);
```

- `pr_notice("Kernel command line: %s\n", saved_command_line);` - эта строка позволяет пользователям видеть, какие параметры были переданы ядру при загрузке, с уровнем важности "notice"



Инициализация механизма "прыжковых меток"

```
jump_label_init();
```

- `jump_label_init()`: инициализирует механизм *jump labels* в ядре Linux, который используется для оптимизации условных ветвлений в коде



Разбор ранних параметров, переданных ядру

```
parse_early_param();
```

- `parse_early_param()`: позволяет настраивать некоторые параметры на ранних этапах загрузки, прежде чем система будет полностью инициализирована



Разбор аргументов командной строки

```
after_dashes = parse_args("Booting kernel",  
                           static_command_line, __start__param,  
                           __stop__param - __start__param,  
                           -1, -1, NULL, &unknown_bootoption);
```

- `after_dashes = parse_args(...):`
 - разбирает аргументы, переданные ядру
 - устанавливает значения для различных переменных, которые будут использоваться в процессе загрузки
 - обрабатывает неизвестные параметры

**Фух! Устали? Ну вот и
закопались.**

Разберем самые интересные функции

Рассмотрим, из всех этих функций самые интересные

- `trap_init()`
 - инициализация механизма обработки исключений и прерываний в ядре
 - инициализация таблиц и обработчиков CPU исключений и прерываний
- `mm_init()`
 - инициализация подсистемы управления памятью
- `sched_init()`
 - инициализация подсистемы планирования задач



Разберем самые интересные функции

- `init_IRQ()`
 - инициализация системы обработки прерываний
 - настройка APIC и прерываний для устройств
- `arch_call_rest_init();`
 - создание начального процесса `init` и потока `kthreadd`
 - настройка некоторых параметров системы
 - внутри эта функция вызывает только `rest_init()`, разберем её подробнее

```

static __initdata DECLARE_COMPLETION(kthreadd_done);

noinline void __ref rest_init(void)
{
    struct task_struct *tsk;
    int pid;

    rcu_scheduler_starting();
    /*
     * We need to spawn init first so that it obtains pid 1, however
     * the init task will end up wanting to create kthreads, which, if
     * we schedule it before we create kthreadd, will OOPS.
     */
    pid = user_mode_thread(kernel_init, NULL, CLONE_FS);
    /*
     * Pin init on the boot CPU. Task migration is not properly working
     * until sched_init_smp() has been run. It will set the allowed
     * CPUs for init to the non isolated CPUs.
     */
    rcu_read_lock();
    tsk = find_task_by_pid_ns(pid, &init_pid_ns);
    tsk->flags |= PF_NO_SETAFFINITY;
    set_cpus_allowed_ptr(tsk, cpumask_of(smp_processor_id()));
    rcu_read_unlock();

    numa_default_policy();
    pid = kernel_thread(kthreadd, NULL, CLONE_FS | CLONE_FILES);
    rcu_read_lock();
    kthreadd_task = find_task_by_pid_ns(pid, &init_pid_ns);
    rcu_read_unlock();

    /*
     * Enable might_sleep() and smp_processor_id() checks.
     * They cannot be enabled earlier because with CONFIG_PREEMPTION=y
     * kernel_thread() would trigger might_sleep() splats. With
     * CONFIG_PREEMPT_VOLUNTARY=y the init task might have scheduled
     * already, but it's stuck on the kthreadd_done completion.
     */
    system_state = SYSTEM_SCHEDULING;

    complete(&kthreadd_done);

    /*
     * The boot idle thread must execute schedule()
     * at least once to get things moving:
     */
    schedule_preempt_disabled();
    /* Call into cpu_idle with preempt disabled */
    cpu_startup_entry(CPUHP_ONLINE);
}

```

Подробный разбор

```
static __initdata DECLARE_COMPLETION(kthreadd_done);
```

`static`: указывает, что переменная имеет внутреннюю ссылку, доступную только в этом файле.

`__initdata`: указывает, что переменная будет использоваться только во время инициализации и может быть освобождена после завершения инициализации.

`DECLARE_COMPLETION(kthreadd_done);` - объявляет переменную `kthreadd_done`, которая используется для синхронизации между потоками. Она будет завершена, когда поток `kthreadd` будет инициализирован.



Подробный разбор

```
noinline void __ref rest_init(void)
```

`noinline`: указывает компилятору не встраивать эту функцию, что может быть полезно для отладки.

`__ref`: указывает, что функция может быть вызвана только из кода, который находится в области памяти, доступной для выполнения.

`rest_init(void)`: определение функции, которая будет вызываться для завершения инициализации ядра.



Подробный разбор

```
struct task_struct *tsk;
```

Объявляет указатель на структуру `task_struct`, которая представляет задачу (процесс) в ядре.

```
int pid;
```

Объявляет переменную `pid`, которая будет использоваться для хранения идентификатора процесса.

```
rcu_scheduler_starting();
```

Подготавливает систему к использованию механизма RCU (Read-Copy Update), который позволяет безопасно обновлять данные, используемые несколькими потоками.



Создание процесса init:

```
pid = user_mode_thread(kernel_init, NULL, CLONE_FS);
```

- Создает новый пользовательский поток, который запускает функцию `kernel_init`
- Этот процесс будет иметь идентификатор `pid == 1`, что делает его основным процессом системы



Закрепление процесса `init` за загрузочным процессором:

```
rcu_read_lock();  
tsk = find_task_by_pid_ns(pid, &init_pid_ns);  
tsk->flags |= PF_NO_SETAFFINITY;  
set_cpus_allowed_ptr(tsk, cpumask_of(smp_processor_id()));  
rcu_read_unlock();
```

`rcu_read_lock();` - блокирует RCU для безопасного чтения данных.

`find_task_by_pid_ns(pid, &init_pid_ns);` - находит задачу по PID и возвращает указатель на структуру `task_struct`

`tsk->flags |= PF_NO_SETAFFINITY;` - устанавливает флаг, который предотвращает миграцию процесса `init` между процессорами



Закрепление процесса `init` за загрузочным процессором:

```
rcu_read_lock();  
tsk = find_task_by_pid_ns(pid, &init_pid_ns);  
tsk->flags |= PF_NO_SETAFFINITY;  
set_cpus_allowed_ptr(tsk, cpumask_of(smp_processor_id()));  
rcu_read_unlock();
```

`set_cpus_allowed_ptr(tsk, cpumask_of(smp_processor_id()));` - устанавливает разрешенные процессоры для процесса `init`, чтобы он выполнялся только на загрузочном процессоре

`rcu_read_unlock();` - освобождает блокировку RCU



Подробный разбор

```
numa_default_policy();
```

Устанавливает политику распределения памяти для систем с поддержкой NUMA (Non-Uniform Memory Access)

Создание потока kthreadd

```
pid = kernel_thread(kthreadd, NULL, CLONE_FS | CLONE_FILES);  
rcu_read_lock();  
kthreadd_task = find_task_by_pid_ns(pid, &init_pid_ns);  
rcu_read_unlock();
```

- создает поток для управления потоками ядра, который будет выполнять функцию kthreadd
- снова используется блокировка RCU для безопасного чтения данных
- включение проверок might_sleep() и smp_processor_id()

Создание потока kthreadd

```
system_state = SYSTEM_SCHEDULING;
```

- устанавливает состояние системы в SYSTEM_SCHEDULING
- позволяет включить проверки, связанные с возможностью блокировки потоков
- завершение инициализации kthreadd

```
complete(&kthreadd_done);
```

- указывает, что инициализация потока kthreadd завершена
- может быть использовано для синхронизации

Запуск планировщика

```
schedule_preempt_disabled();
```

Вызывает планировщик, чтобы запустить выполнение задач.
Это необходимо для того, чтобы система начала обрабатывать задачи.

```
cpu_startup_entry(CPUHP_ONLINE);
```

Вызывает функцию, которая переводит процессор в состояние ожидания, позволяя ему обрабатывать прерывания и выполнять другие задачи.



Небольшой вывод

Инициализация ядра является не менее сложным процессом, чем его загрузка.

Прежде чем на экране современного компьютера появится сообщение "Hello, world", ядро выполняет огромный объем работы, связанной с аппаратным обеспечением, различными механизмами безопасности и настройкой определенных подсистем.



Основные функции ядра Linux

Управление процессами

Создание и завершение процессов

- Ядро Linux управляет процессами, создавая и завершая их по мере необходимости.
 - Процесс **создается** с помощью системного вызова `fork()`, который создает копию текущего процесса.
 - **Новый процесс**, называемый дочерним, получает уникальный идентификатор процесса (PID).
 - После создания дочерний процесс может вызвать `exec()` для замены своего адресного пространства на новое, загружая другую программу.



Управление процессами

Создание и завершение процессов

- **Завершение** процесса происходит через системный вызов `exit()`
 - Он освобождает ресурсы, связанные с процессом, и уведомляет родительский процесс о завершении.
 - Родитель может использовать `wait()` для ожидания завершения дочернего процесса и получения его статуса.



Управление процессами

Планирование процессов

- Планировщик процессов в ядре Linux отвечает за распределение процессорного времени между активными процессами.
- Он использует различные алгоритмы планирования, такие как Completely Fair Scheduler (CFS), который обеспечивает справедливое распределение времени CPU между процессами.
- Планировщик принимает во внимание приоритеты процессов, их состояние (готов, выполняется, заблокирован) и другие факторы, чтобы оптимизировать производительность системы.



Управление памятью

Аллокация и освобождение памяти

- Ядро Linux управляет физической и виртуальной памятью, выделяя и освобождая память по мере необходимости.
 - Для аллокации памяти используются различные механизмы, такие как buddy allocator и slab allocator.
 - Эти механизмы обеспечивают эффективное использование памяти, минимизируя фрагментацию и ускоряя выделение.
- Функции, такие как `kmalloc()` и `kfree()`, используются для выделения и освобождения памяти в ядре.
 - Ядро также поддерживает механизмы отладки памяти, которые помогают выявлять утечки и ошибки в управлении памятью.



Управление памятью

Виртуальная память

- Виртуальная память позволяет каждому процессу иметь собственное адресное пространство, что обеспечивает изоляцию и безопасность.
- Ядро использует таблицы страниц для сопоставления виртуальных адресов с физическими.
- Это позволяет процессам использовать больше памяти, чем доступно физически, благодаря механизму подкачки (paging), который перемещает данные между оперативной памятью и диском.



Управление устройствами

Драйверы устройств

- Модули ядра, которые обеспечивают взаимодействие между аппаратным обеспечением и программным обеспечением
- Отвечают за управление устройствами, такими как диски, сетевые интерфейсы и графические карты

Ядро Linux поддерживает множество драйверов, которые могут быть загружены и выгружены динамически, что позволяет добавлять поддержку новых устройств без перезагрузки системы.



Управление устройствами

Ввод/вывод

Ядро Linux

- управляет операциями ввода/вывода (I/O) через абстракции, такие как файловые дескрипторы и системные вызовы
- обрабатывает запросы на чтение и запись данных
- управляет буферизацией и кэшированием
- обеспечивает асинхронный ввод/вывод для повышения производительности



Управление устройствами

Обработка прерываний и сигналов

- Ядро обрабатывает прерывания, которые возникают от аппаратных устройств, таких как клавиатура или сетевой интерфейс
 - Когда устройство генерирует прерывание, ядро временно приостанавливает выполнение текущего процесса и выполняет обработчик прерывания, который выполняет необходимые действия (например, чтение данных)
- Сигналы — это механизм межпроцессного взаимодействия, который позволяет процессам отправлять уведомления друг другу
 - Ядро обрабатывает сигналы, такие как **SIGINT** (прерывание) и **SIGTERM** (завершение), позволяя процессам реагировать на события



Системные вызовы и интерфейсы

- Системные вызовы — это интерфейсы, через которые пользовательские приложения взаимодействуют с ядром
 - Они предоставляют функции для выполнения операций, таких как создание процессов, управление памятью, работа с файлами и сетевыми соединениями
 - Примеры системных вызовов включают `open()`, `read()`, `write()`, `fork()`, `exec()`, и `exit()`
- Ядро предоставляет API для разработки приложений
 - Позволяет разработчикам использовать функции ядра без необходимости взаимодействовать с аппаратным обеспечением напрямую
 - Обеспечивает уровень абстракции, который упрощает разработку и повышает безопасность



Вопросы



если есть вопросы



если вопросов нет

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋

Приглашаем на следующий вебинар: “0x04 Модуль ядра. Как его написать, собрать, запустить.”



Ссылка на вебинар будет
в ЛК за 15 минут



Материалы
к занятию в ЛК —
можно изучать



Обязательный материал
обозначен красной лентой

Подготовка к следующему занятию

- 1) Вспомнить что такое Makefile и как их писать.
- 2) Развернуть систему, для запуска модулей ядра.

обсуждаем Off-topic

***Время отключить запись**

OTUS

